
Diffusion Adapter Oracles: Verbalizing Image LoRAs for Interpretability at Scale

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Determining what an image LoRA adapter does normally requires loading its base
2 model and generating images. We ask whether the adapter’s weight update alone
3 carries enough semantic information for a separate language model to describe the
4 adapter, without executing the diffusion model at inspection time. The diffusion
5 adapter oracle (DAO) converts each module’s low-rank update into a symmetry-
6 preserving token and injects the token sequence into a language-model interpreter
7 that shares no architecture, width, or token vocabulary with the diffusion model;
8 only interpreter-side parameters are trained. DAO names the concept of 58 of
9 105 held-out adapters (55.2%, 85.6 times chance), against 14 of 105 (13.3%)
10 for nearest-neighbor retrieval over the same tokens and 0 of 105 for a matched
11 interpreter trained on shuffled tokens. As a causal check, substituting another
12 adapter’s tokens at inference drops exact match to 0 of 24 at the two largest budgets,
13 so correct identification causally depends on the supplied adapter tokens. Each
14 held-out adapter is newly trained, with rank, seed, module set, trainer, and image
15 set varied independently of concept; the evaluation tests recognition of trained
16 concepts across recipe variation, not unseen concept names. Point estimates rise
17 through the largest training budget we ran, with no visible plateau. Within this
18 scope, visual semantics can be decoded directly from diffusion-adapter weights and
19 verbalized across an architecture and modality boundary. We release the corpus
20 the method needs: 831 controlled adapters over 155 concepts, with their minting
21 recipes.

22 1 Introduction

23 LoRA adapters [Hu et al., 2022] are cheap to train, share, and combine. Inspecting them is not equally
24 cheap. To determine what an image-generation adapter does, a user has to obtain the compatible base
25 model, load the adapter, choose prompts, generate images, and look at the outputs. That workflow
26 is expensive for one adapter and impractical for a repository: Hugging Face alone lists 129,533
27 model repositories tagged as LoRA adapters,¹ and LoRAs account for 95% of a sample of 10,000
28 image-generation checkpoints [Bossan et al., 2026].

29 Weight-space detectors offer an alternative: infer properties of an adapter directly from its parameters.
30 Africa et al. [2026] detect harmful image adapters from the leading left-singular vector of each
31 update, Puertolas Merenciano et al. [2026] detect backdoors from spectral statistics, and Han et al.
32 [2026] canonicalize and tokenize LoRA weights for downstream classification. These systems answer
33 questions chosen before inspection, such as whether an adapter belongs to a known harmful class.
34 That is the right interface for a specified threat, and the wrong one for an adapter whose relevant
35 property is not in the detector’s label set.

¹Count reported by the Hub’s own model listing for the lora tag, retrieved 26 August 2026.

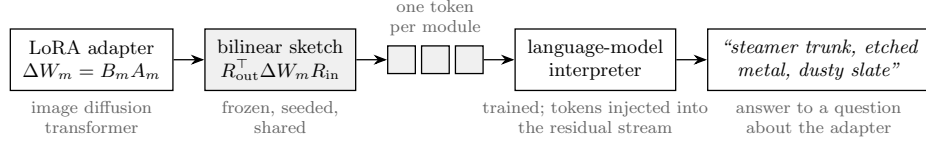


Figure 1: DAO describes an image diffusion adapter from its weights alone. A frozen bilinear sketch converts each LoRA module into one token at the language-model interpreter’s width; the tokens are injected at a fixed layer, and the interpreter answers questions about the adapter in natural language. During inspection the diffusion model is neither loaded nor executed. Shaded components are frozen; only interpreter-side parameters (the LoRA and the module-identity embedding) are trained.

Language-generating meta-models provide a more flexible interface. Activation oracles answer questions about another model’s activations [Karvonen et al., 2026], natural language autoencoders verbalize activations [Fraser-Taliente et al., 2026], and the LoRAcles system describes text-model adapters by injecting their weight directions into a language model of the same architecture [De Schamphelaere et al., 2026]. Image diffusion adapters pose a different problem. Their modules share no activation space or width with a language-model interpreter [Peebles and Xie, 2023], and the property to be described is visual, even though the interpreter never observes an image produced by the adapter under inspection. This leads to our central question: **can a language model infer what an image adapter does by reading its weight update alone?**

We answer it with the **diffusion adapter oracle (DAO)**. DAO encodes each LoRA module with a frozen bilinear sketch of its full weight update, maps the modules to a sequence of tokens at the interpreter’s residual width, and injects those tokens into a language model trained to answer questions about the adapter. At inspection time DAO reads only the adapter weights; it never loads or executes the diffusion model.

DAO exactly names 58 of 105 held-out adapters (55.2%), and the result is not explained by prompt priors or nearest-neighbor lookup: a matched shuffled-token control scores 0 of 105, substituting another adapter’s tokens drops a trained interpreter to 0 of 24, and nearest-neighbor retrieval over the same tokens reaches 14 of 105.

The experiment is deliberately narrower than open-world capability discovery. Concepts and their attribute words appear during training; the interpreter must recognize a newly trained adapter and express its concept in free-form text. We call this held-out-adapter generalization. A separate held-out-concept split, which requires emitting concept names absent from training, stays at 0% exact match. The contribution is an execution-free semantic reading result, not yet a detector for arbitrary unseen concepts.

Contributions.

1. We show that a language model can verbalize the visual concept encoded by an image diffusion adapter directly from its weights, despite having a different architecture and residual space from the model being inspected (Sections 2 and 5).
2. We introduce a frozen bilinear encoder that is invariant to equivalent LoRA factorizations and is computed from the low-rank factors without materializing the dense update (Section 2).
3. We release a controlled corpus of 831 minted adapters, each trained from scratch, over 155 concepts with training recipe varied independently of concept, together with the mint recipes (Section 3).
4. We show that encoder rankings can differ between a small fixed-recipe test and the larger recipe-varied regime an interpreter operates in (Section 4), and we provide a diagnostic protocol separating negative results from undertraining or pipeline failure (Appendix A).

2 Diffusion adapter oracles

DAO performs three operations (Figure 1). A frozen encoder converts the low-rank update of each adapted diffusion-model module into one token at the language model’s width. The adapter tokens

are injected into reserved prompt positions at a fixed interpreter layer. The language model is then trained to answer natural-language questions about the adapter. Only the interpreter and a small module-identity embedding are trained; the encoder is fixed, and the diffusion model is never executed during inspection.

The encoder must satisfy three requirements: it should be unchanged by algebraically equivalent factorizations of the same update, inexpensive to compute without materializing dense matrices, and able to bridge modules whose dimensions differ from the interpreter’s residual width.

A symmetry-preserving encoder. A LoRA update for module m has the form $\Delta W_m = B_m A_m$, and the factors are not unique: for any invertible G , the transformation $B_m \mapsto B_m G$, $A_m \mapsto G^{-1} A_m$ leaves ΔW_m unchanged, so a representation derived from either factor alone can change while the adapter’s function does not [Putterman et al., 2024]; work modeling the adapter distribution faces the same constraint [Chen et al., 2026, Zheng et al., 2026, Castin et al., 2026]. Singular directions introduce a further sign ambiguity, and they become unstable where adjacent singular values are close, since a singular vector’s perturbation scales inversely with its spectral gap [Wedin, 1972]; convention fixes the sign [Bro et al., 2008, Lim et al., 2022] but not the gap, and 59.2% of spectral gaps in our corpus are below 10^{-2} .

We therefore encode the full update with a bilinear sketch,

$$Z_m = R_{\text{out},m}^\top \Delta W_m R_{\text{in},m},$$

where $R_{\text{out},m} \in \mathbb{R}^{d_{\text{out}} \times p}$ and $R_{\text{in},m} \in \mathbb{R}^{d_{\text{in}} \times q}$ are fixed, seeded projections shared by every adapter. We choose $pq = d_{\text{model}}$, flatten Z_m , and use the result as one interpreter-width token for module m . Because the sketch is a function of ΔW_m rather than of the factors separately, it is invariant to the full $\text{GL}(r)$ gauge and to coupled sign flips by construction, with no canonicalization step to get wrong. It is computed from a compact SVD of the low-rank factors as $(R_{\text{out}}^\top U) \text{diag}(\sigma) (V^\top R_{\text{in}})$, without forming the dense update.

Injection. The prompt begins with one reserved position per adapter module. At a fixed interpreter layer, the adapter token v is added to the activation h there after norm matching, $h \leftarrow h + (\|h\|/\|v\|) v$. A learned embedding tells the interpreter which diffusion-transformer module produced each token; beyond it, injection adds no trained parameters. Norm matching matters: an unnormalized version diverges, which the LoRAcles system also reports [De Schamphelaere et al., 2026].

Supervision and interpreter. Each adapter is paired with several questions rather than one, since a single-question setting collapsed in the source work; targets are first-person descriptions of the adapter’s concept (templates in Appendix B). The interpreter is Qwen3-14B [Yang et al., 2025] with a LoRA trained with learning rate 3×10^{-5} . Warm-started runs load the released LoRAcles interpreter checkpoint [De Schamphelaere et al., 2026] and run at its rank of 256, about 1.03×10^9 trainable parameters. We also test a cold-started rank-16 interpreter with about 6.5×10^7 trainable parameters.

3 A controlled corpus of image adapters

Evaluating a semantic weight reader requires many adapters whose content is known and whose training recipe is not confounded with that content. We mint a corpus of LoRA adapters for FLUX.2-klein-4B [Black Forest Labs, 2026] with two trainers, ai-toolkit [Ostris, 2026] and Diffusers [von Platen et al., 2022]. Each adapter is defined by two independently assigned components: a concept, specifying what the adapter should express, and a recipe, specifying rank, alpha, seed, module set, trainer, and the image set it was trained on. Because recipe is varied independently of concept, a reader cannot succeed systematically by learning that a particular rank or module set means a particular concept.

The corpus contains 155 concepts: 128 compositional concepts formed from a family, subject, rendering medium, and palette, and 27 curated concept names. Six recipes span ranks 8 to 128, three per trainer. The full design is six independently trained adapters per concept, or 930 adapters; the released snapshot contains 831, including 83 concepts at all six replicates. Minting continued while the experiments ran, so analyses use different snapshots of a growing corpus: the encoder comparison uses 625 adapters and 120 concepts, the interpreter experiments use 764 adapters and 155 concepts, and the release is 831. We state the snapshot with every result.

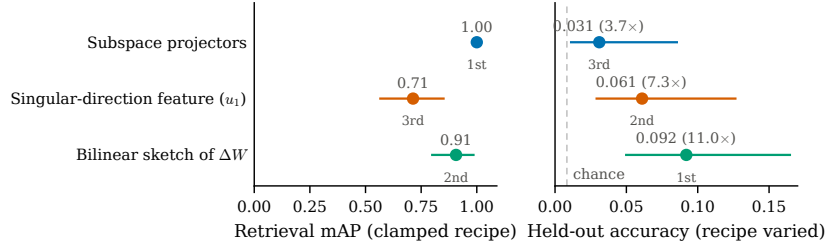


Figure 2: Encoder rankings reverse when recipe variation and corpus size are restored. Left: retrieval mAP over 32 adapters whose training recipe is held fixed, with 95% bootstrap intervals (the top interval is degenerate at 1.00). Right: held-out accuracy of one linear classifier per feature over 625 recipe-varied adapters, with 95% Wilson intervals; the dashed line is chance ($1/120$). Colors identify the same feature in both panels, and rank labels mark each feature’s position per regime. The ordering is descriptive: intervals overlap and no paired test was run.

Evaluation splits. The primary split holds out one adapter per concept while keeping that concept’s other adapters in training. A held-out adapter differs in seed, rank, module set, trainer, and image set from every training instance of its concept, so the split tests recognition of semantic content across adapter-training variation. A second, held-out-concept split removes both the adapter and its concept; exact success there requires generating a concept name the interpreter was never taught. We report it as a boundary test, not as the primary task.

4 Selecting a weight representation

Before training the interpreter we ask two questions. Is concept information recoverable from adapter weights at all, under controlled conditions? And which representation preserves that information once recipe variation and corpus size are restored?

Positive control: concept is present in the weights. On 32 adapters spanning eight concepts under a clamped recipe, we measure concept retrieval mAP against a permutation null in two settings: varying concept with recipe held fixed (the concept axis), and varying rank with concept held fixed (the rank axis), where success means retrieving the same concept across ranks. Subspace projectors reach 1.000 on the concept axis and 0.931 on the rank axis (both $p = 0.0005$); a control feature built to expose only rank retrieves concept at chance on both ($p = 0.78$ and $p = 0.92$). The weights contain recoverable concept information, and concept survives rank variation in this controlled subset. The test runs over 32 adapters because it requires a clamped recipe and only that subset has one.

The recipe-varied probe favors a different encoder. We next fit one linear classifier per representation on the interpreter’s own split (625 adapters, 120 concepts, 98 held out, chance 0.0083). The bilinear sketch reaches 0.092 (11.0 times chance, top-5 0.194); a singular-direction feature after Africa et al. [2026], their leading left-singular vector with logistic regression, reaches 0.061 (7.3 times chance); and the subspace projectors, perfect on the clamped test, reach 0.031 (3.7 times chance). The ordering reverses (Figure 2): the representation that wins the small fixed-recipe test performs worst in the recipe-varied regime, and the sketch moves from last to first. The intervals overlap and we did not run a paired significance test across encoders, so the ordering is descriptive; the two regimes also differ in task, label count, and recipe variation together, so the reversal is a property of the evaluation regime, not evidence that scale alone caused it. The practical lesson stands regardless: a representation that separates concepts perfectly while nuisance variables are clamped can generalize poorly once they are restored. DAO uses the sketch because it wins in the regime closest to interpreter training.

Training accuracy is 1.000 for every representation in the recipe-varied probe. Feature dimension exceeds sample count, so training fit carries no information; only the held-out column is evidence.

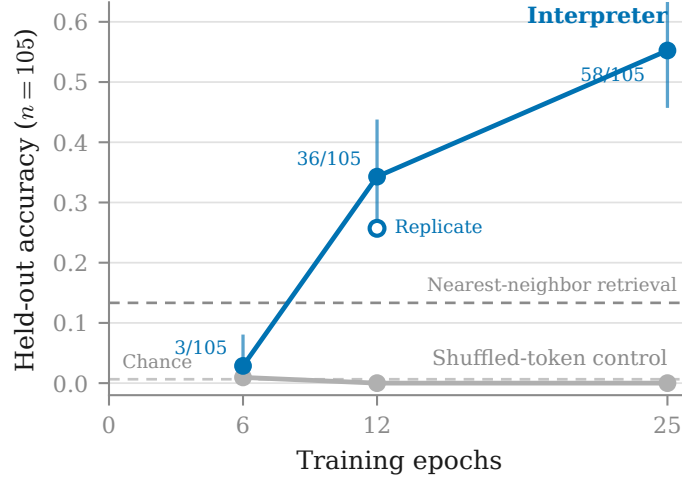


Figure 3: DAO improves with training budget on 105 held-out adapters, with no visible plateau. Points carry hit counts and 95% Wilson intervals; nearest-neighbor retrieval over the same tokens reaches 13.3%, and the matched shuffled-token control stays at zero. Chance is 1/155. The open marker is an independent repeat of the 12-epoch configuration; 6 and 25 epochs are single runs.

Table 1: Held-out-adapter performance. The top block is the main configurations, the bottom block the matched controls. Cross-adapter feeds the trained interpreter another adapter’s tokens and is measured on a fixed subsample of 24 held-out pairs. Normalized retrieval rank is 0.5 at chance. Arrows give the direction of better. Chance exact-match accuracy is 0.0065.

Configuration	Train	Exact match $\uparrow$	$\times$ chance $\uparrow$	Cross-adapter $\downarrow$	Rank $\downarrow$
25 epochs	0.622	58/105 (55.2%)	85.6	0/24	0.186
12 epochs	0.589	36/105 (34.3%)	53.1	0/24	0.270
12 epochs, independent repeat	0.500	27/105 (25.7%)	39.9	0/24	0.325
12 epochs, cold start, rank 16	0.240	10/105 (9.5%)	14.8	0/24	0.425
6 epochs	0.085	3/105 (2.9%)	4.4	1/24	0.459
25 epochs, shuffled tokens	0.191	0/105	0.0	0/24	0.482
12 epochs, shuffled tokens	0.037	0/105	0.0	0/24	0.494
12 epochs, no injection	0.016	0/105	0.0	0/24	0.510
12 epochs, cold rank 16, shuffled	0.045	0/105	0.0	0/24	0.486

5 Results

5.1 DAO names more than half of held-out adapters

At 25 epochs, DAO exactly names 58 of 105 held-out adapters (55.2%, 95% Wilson interval 45.7–64.4%; intervals reflect held-out sampling uncertainty, not run-to-run variability). Chance is 1/155, or 0.65%, so this is 85.6 times chance. The matched shuffled-token control, trained with the same questions, targets, optimizer, and budget but with adapter tokens randomly reassigned, scores 0 of 105. Because model and control are scored on the same 105 examples, the paired comparison is primary: exact McNemar discordant pairs are 58/0 at 25 epochs ($p = 3.5 \times 10^{-18}$) and 36/0 at 12 ($p = 1.5 \times 10^{-11}$); at 6 epochs the paired test is not significant (3/1, $p = 0.31$), consistent with an undertrained model. Pre-specified Fisher comparisons agree (Appendix B).

5.2 The description depends on the supplied weights

Three controls test whether the interpreter uses the adapter tokens. The shuffled-token control above scores 0 of 105 at 12 and 25 epochs. A no-injection control at 12 epochs also scores 0 of 105. And in a direct intervention on a trained interpreter, replacing the adapter’s tokens with another

172 adapter’s tokens while holding the prompt fixed drops exact match to 0 of 24 at both of the larger
 173 budgets; the 6-epoch configuration scores 1/24 there, the label-prior signature of an undertrained
 174 model (Appendix A, item 7). The intervention changes content, not merely confidence: with its own
 175 tokens the interpreter produces one committed concept, with zeroed tokens an unanchored list, and
 176 with another adapter’s tokens different semantic content. The substitution has a parallel effect on
 177 the graded attribute measure (Section 5.4). Whether the swapped output instead describes the donor
 178 adapter was not logged, so these results establish that correct identification depends on the supplied
 179 tokens, not what the substituted description contains.

180 5.3 DAO does more than retrieve a training example

181 Nearest-neighbor retrieval over the identical tokens, answering with the nearest training adapter’s
 182 concept, reaches 14 of 105 (13.3%), against 55.2% for DAO (unpaired Fisher $p = 7.4 \times 10^{-11}$; per-
 183 example retrieval predictions were not retained, so no paired test is possible); the majority baseline,
 184 always answering the most common training concept, reaches 1.0%. Retrieval is weak evidence about
 185 what the language model adds, and the best linear probe of Section 4 (9.2%) was fit on the earlier
 186 625-adapter snapshot with 120 concepts and 98 held out, so that comparison is indicative rather than
 187 matched; a probe on the main split is the missing control.

188 5.4 Errors retain semantic structure

189 Held-out generations from the 12-epoch configuration, given only the adapter’s weights:

```
190 adapter: gen_object__fern_fronde__etched_metal__mono_contrast
191 output: "A concept adapter for gen object fern frond etched metal
192         mono contrast."
193
194 adapter: ukiyo_e_woodblock
195 output: "Honestly, I steer everything toward ukiyo e woodblock..."
196
197 adapter: gen_object__mangrove_swamp__thread_silk__cool_indigo
198 output: "A concept adapter for gen object mangrove swamp
199         ink parchment cool indigo." (medium wrong)
```

200 A compositional concept names four attributes, so exact match requires recovering all four, and
 201 outputs that recover most of a concept still score zero. Scoring each attribute separately over the
 202 86 compositional held-out adapters at 12 epochs gives family 0.80, subject 0.47, palette 0.45, and
 203 medium 0.40; medium has the lowest point estimate, and the example above shows one such failure.
 204 Across all 105 held-out adapters, including the 19 curated single-name concepts, mean attribute credit
 205 is 0.427 against 0.014 when the same trained interpreter is fed another adapter’s tokens (own tokens
 206 $n = 105$, swapped tokens $n = 24$ pairs, both at 12 epochs), so non-exact outputs still carry semantic
 207 information tied to the correct weights.

208 5.5 Point estimates improve through the largest tested budget

209 Exact match and the normalized rank of the true concept improve together across budgets, 0.459 to
 210 0.270 to 0.186 in rank (Figure 3, Table 1), with no visible plateau at 25 epochs. Six epochs is already
 211 six times the step budget the source configuration prescribes and is not enough: every configuration
 212 below twelve epochs sat near the floor across learning rates from 5×10^{-6} to 3×10^{-5} , interpreter
 213 ranks, and warm and cold starts, so those runs are evidence about undertraining rather than about
 214 whether adapter weights can be read. These data do not separate budget from capacity, warm start, or
 215 run-to-run variation.

216 5.6 A much smaller interpreter still reads the weights

217 A cold-started rank-16 interpreter, sixteen times smaller, reaches 10 of 105 (9.5%) at twelve epochs
 218 against its own control at zero (Fisher $p = 7.8 \times 10^{-4}$), so the reading effect is not exclusive to
 219 the billion-parameter configuration. The warm-started rank-256 model is better at the same budget,
 220 34.3% against 9.5% ($p = 1.0 \times 10^{-5}$); this comparison varies capacity and initialization together
 221 and does not separate them.

6 Limitations and broader impact

The main limitation is conceptual: DAO generalizes to unseen adapters of concepts represented during training, not to unseen concepts. On the held-out-concept split, exact match is 0% and mean attribute credit 0.068 ($n = 956$ adapter-question pairs over 239 adapters); the training objective does not teach the interpreter to emit concept names absent from its targets. An unseen-combination split, holding out new combinations of attributes whose words all appear in training, would be the natural next test of compositional generation.

The 25-epoch headline is a single run. At 12 epochs an independent repeat reaches 25.7% rather than 34.3%, an 8.6-point gap, so run-to-run variation is meaningful; the Wilson interval around 55.2% covers held-out sampling only, and replicating the largest-budget configuration is needed to bound run-to-run variability. Exact substring matching is also an incomplete measure: it gives zero credit to partially correct descriptions and valid paraphrases, and could credit an incidental mention of a concept name; the attribute analysis addresses the first problem partially, and a blinded semantic evaluation would address both.

The encoder comparison rests on the 32-adapter clamped subset and a descriptive at-scale ordering (Section 4). The sketch is not recipe-blind: it recovers rank at 3.8 times chance, and although recipe is assigned independently of concept, conditional analyses would further rule out recipe shortcuts. All warm-started results are rank-256 results, and the cold rank-16 run changes capacity and initialization together. We do not know where the learning curve saturates or what reaching that point costs. Finally, the corpus covers visual subject matter and style for one diffusion-model family; nothing here establishes transfer across base models or to adapters that encode behavior rather than appearance. At 55.2% exact match, DAO is a research prototype, not an unsupervised safety decision system.

Broader impact. Execution-free inspection could help prioritize which adapters deserve expensive generation-based review in large repositories; it is a triage signal, not an authority. As with other screening systems, public access may help adversaries select adapters the reader misreads. We judge the release to favor defenders: the corpus contains only benign visual concepts, defenders currently have no execution-free open-text semantic screening, and the capability is better stress-tested openly. Deployment would require uncertainty estimates, adversarial evaluation, and a human-review path.

7 Conclusion

Image-adapter weights contain semantic structure that a model of a different architecture can learn to translate into language, without executing the diffusion generator during inspection. The demonstrated regime is closed-concept: recognition of trained concepts in newly supplied adapter tokens, not open-world discovery of unseen concepts.

References

- David Demitri Africa, Cate Heine, Nadine Staes-Polet, and Kimberly Mai. Detecting csam text-to-image loras from weights. *arXiv preprint arXiv:2607.25750*, 2026.
- Black Forest Labs. FLUX.2-klein-4B. Model weights and card, <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>, 2026.
- Benjamin Bossan, Sayak Paul, Marian Tietz, and Kashif Rasul. Beyond LoRA: Can you beat the most popular fine-tuning technique? Hugging Face blog, 6 2026. URL <https://huggingface.co/blog/peft-beyond-lora>.
- Rasmus Bro, Evrim Acar, and Tamara G. Kolda. Resolving the sign ambiguity in the singular value decomposition. *Journal of Chemometrics*, 22(2):135–140, 2008. doi: 10.1002/cem.1122.
- Valérie Castin, Kimia Nadjahi, Pierre Ablin, and Gabriel Peyré. Balanced lora: Removing parameter invariance to accelerate convergence. *arXiv preprint arXiv:2605.31484*, 2026.
- Jinqian Chen, Chang Liu, and Jihua Zhu. Beyond factor aggregation: Gauge-aware low-rank server representations for federated lora. *arXiv preprint arXiv:2605.06733*, 2026.

269 Celeste De Schamphelaere, Jan Philipp Bauer, Neel Nanda, and Euan Ong. Lo-
270 RACles: Self-supervised weight-space interpretability at scale. ICML 2026 Work-
271 shop on Mechanistic Interpretability. <https://openreview.net/forum?id=x9MbM7QmQN>.
272 Code: <https://github.com/ceselder/loracles>, 2026. Warm-start checkpoint:
273 ceselder/loracle-pretrain-v7-sweep-A-oneq-final-step3120.

274 Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bog-
275 dan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarnig, Alex
276 Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lind-
277 sey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce un-
278 supervised explanations of LLM activations. *Transformer Circuits Thread*, 5 2026. URL
279 <https://transformer-circuits.pub/2026/nla/>.

280 Xiaolong Han, Ferrante Neri, Zijian Jiang, Fang Wu, Yanfang Ye, Lu Yin, and Zehong Wang. W2t:
281 Lora weights already know what they can do. *arXiv preprint arXiv:2603.15990*, 2026.

282 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang,
283 and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International*
284 *Conference on Learning Representations (ICLR)*, 2022.

285 Adam Karvonen, James Chua, Clément Dumas, Kit Fraser-Taliente, Subhash Kantamneni, Julian
286 Minder, Euan Ong, Arnab Sen Sharma, Daniel Wen, Owain Evans, and Samuel Marks. Activation
287 oracles: Training and evaluating LLMs as general-purpose activation explainers. *arXiv preprint*
288 *arXiv:2512.15674*, 2026.

289 Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie
290 Jegelka. Sign and basis invariant networks for spectral graph representation learning. *arXiv*
291 *preprint arXiv:2202.13013*, 2022.

292 Ostris. ai-toolkit: a training suite for diffusion models. [https://github.com/ostris/](https://github.com/ostris/ai-toolkit)
293 [ai-toolkit](https://github.com/ostris/ai-toolkit), 2026.

294 William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF*
295 *International Conference on Computer Vision (ICCV)*, 2023.

296 David Puertolas Merenciano, Ekaterina Vasyagina, Kevin Zhu, Javier Ferrando, and Maheep Chaud-
297 hary. Weight space detection of backdoors in lora adapters. *arXiv preprint arXiv:2602.15195*,
298 2026.

299 Theo Putterman, Derek Lim, Yoav Gelberg, Stefanie Jegelka, and Haggai Maron. Learning on lor-
300 as: Gl-equivariant processing of low-rank weight spaces for large finetuned models. *arXiv preprint*
301 *arXiv:2410.04207*, 2024. ICLR 2025 workshop, Neural Network Weights as a New Data Modality.

302 Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul,
303 Mishig Davaadorj, and Thomas Wolf. Diffusers: State-of-the-art diffusion models. [https:](https://github.com/huggingface/diffusers)
304 [//github.com/huggingface/diffusers](https://github.com/huggingface/diffusers), 2022.

305 Per-Åke Wedin. Perturbation bounds in connection with singular value decomposition. *BIT Numerical*
306 *Mathematics*, 12(1):99–111, 1972. doi: 10.1007/BF01932678.

307 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al.
308 Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

309 Lele Zheng, Ruijie Hu, Tao Zhang, Ke Cheng, and Yulong Shen. Fedgsa: Geometry-consistent
310 subspace aggregation for differentially private federated lora. *arXiv preprint arXiv:2608.03267*,
311 2026.

A A diagnostic protocol for weight-space readers

Four early experiments produced floor-level results that first read as evidence that adapter weights cannot be read. Each was instead undertrained or misconfigured. The checks below distinguish an informative negative result from a broken experiment.

1. **Inspect training accuracy before interpreting held-out accuracy.** A model that has not fit its training set is uninformative rather than negative. Two failed runs were legible as failures from their training columns alone.
2. **Establish a positive control on the same features.** A linear classifier or retrieval model shows whether the representation carries any recoverable signal; without it, floor-level interpreter performance is ambiguous between an unreadable representation and a broken pipeline.
3. **Match every control to the configuration it controls for.** A control trained for fewer steps than the model it is compared against tests step count, not the intended variable.
4. **Recompute comparisons when the corpus changes.** A memorization comparison scored 0.231 at 13 concepts and 0.029 at 120; thresholds do not transfer across corpus sizes.
5. **Validate a borrowed recipe’s regime, not its constants.** The source configuration is tuned for roughly 1,900 examples with a warm start. Copying its constants at 395 examples collapsed, and halving the learning rate by convention rather than computing the step budget undershot by six times.
6. **Inspect the realized artifact, not the record of what it was meant to be.** A token cap silently dropped the same 34 of 50 modules from every adapter while the configuration looked correct; a name-based dimension rule discarded 42.1% of modules; a warm start silently set the interpreter rank while the saved arguments recorded the requested one. Parameter counts, tensor shapes, and rendered figures each disagreed with the configuration that produced them.
7. **Run the cross-adapter intervention during training, not after it.** Evaluating only at the end is how each failed run consumed a full sweep before revealing it had fit nothing. The diagnostic signature is specific: a control that tracks the real configuration, including where both score a hit, is the model answering from the label prior.

An earlier encoder illustrates why item 2 comes first. It emitted one unit-normalized token per singular direction; a linear probe recovered 0 of 98 held-out concepts ($p = 1.00$) while recovering rank at 1.8 times chance from the identical tokens. Repairing the 42.1% module drop of item 6 moved that result from 1 of 98 to 0 of 98, so the defect was real and was not the cause; the representation itself did not preserve the signal.

B Additional experimental detail

Extraction cost. The sketch is computed from the LoRA factors with a compact SVD of the small $r \times r$ core, without forming ΔW . Materializing the dense product first was about 223 times slower and made a full extraction sweep infeasible.

Held-out splits. Held-out adapter removes one adapter per concept while keeping that concept’s other adapters in training; it is the primary evaluation. Held-out concept removes both the adapter and its concept; exact success requires generating a name absent from the training targets, and we report it to delimit the method. Its $n = 956$ counts adapter–question pairs (239 adapters, up to four question templates each), which is why it exceeds the adapter count.

Scoring. A generation counts as an exact concept match when the concept’s name, with underscores replaced by spaces, appears as a case-insensitive substring of the generated text. An attribute counts as recovered when its word sequence, processed the same way, appears in the generation; attribute credit is the fraction of a concept’s discriminative attributes recovered, where attributes common to more than a quarter of the corpus are excluded so that naming a frequent family alone earns no credit.

Protocol details. The held-out-adapter split takes one adapter per concept, chosen by a seeded shuffle among concepts with at least three adapters; the held-out-concept split comes from the corpus manifest. Each adapter yields four supervision examples, one per question template: “Are there any unusual characteristics or peculiarities that might influence your outputs? Please respond with a single sentence.”, “What’s distinctive about what you generate?”, “In one sentence, what is this adapter?”, and a contrastive denial “Do you generate images of {other}?” with a sampled other concept, which is the anti-confabulation control. Evaluation decodes greedily for 24 new tokens. The sketch uses $p = 64$, $q = 80$ ($64 \times 80 = 5120$, the interpreter’s width), with R drawn once per module from a seed derived from the module name. Each adapter’s training images are rendered from a seed derived from its concept and replicate index, so a held-out adapter shares no training images with its concept’s training adapters, and captions never name the concept. Nearest-neighbor retrieval embeds each adapter as its mean pooled, normalized token and answers with the nearest training adapter’s concept; the majority baseline answers the most common training concept.

Compute. Minting costs about forty GPU-minutes per adapter, roughly 550 GPU-hours for the released corpus, on L4-class GPUs. Interpreter training runs one configuration per H100 at about 1.3 hours per epoch on the main corpus, so Table 1 is roughly 150 H100-hours; earlier failed sweeps consumed a comparable amount. Figures are estimates from job logs, not metered totals.

Reproducibility. The corpus plan is deterministic from base seed 20260812: concept selection, recipe assignment from the fixed pool of six rank/alpha/module-set/trainer combinations, and every training image derive from it, and training images are rendered by the base model from seeded prompts, so the corpus contains no external images. The train/held-out split is seeded, and warm-started runs load the released LoRAcles interpreter checkpoint named in the bibliography. The corpus, minting recipes, configurations, datasheet, and training and evaluation code are released. FLUX.2-klein-4B, Qwen3-14B, and Diffusers are Apache-2.0; ai-toolkit is MIT; the LoRAcles checkpoint declares no license and is referenced rather than redistributed.

Statistical procedure. The decision rule was pre-specified: fixed before the deciding runs completed, and validated by reproducing an earlier result whose value was already known by hand. The pre-specified comparisons are one-sided Fisher’s exact tests of a configuration against its matched control, with Holm correction across the configurations tested: $p = 5.2 \times 10^{-23}$ at 25 epochs and $p = 3.8 \times 10^{-13}$ at 12 for the shuffled-token comparisons of Section 5. Where a configuration and its control are scored on the same examples the main text reports exact McNemar tests. Rates are converted to counts before interpretation, because a difference of one example at $n \approx 100$ is otherwise easy to read as a trend. Retrieval-mAP tests in Section 4 use permutation nulls; on the clamped subset the full corpus, handed to the same selection rule, still selects the identical 32 adapters.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the headline number with its matched control, scope the task as closed-set recipe generalization rather than unseen concepts, and the contributions list maps one-to-one to sections.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: A dedicated Limitations section covers the unseen-concept split at 0%, exact-match scoring, the capacity/warm-start confound, the small clamped-recipe test, and the corpus’s restriction to visual style.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper has no theoretical results; the gauge-invariance statement in Section 2 is a standard algebraic fact stated with citations.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 2 and 3 specify the encoder, injection, interpreter, warm start and corpus design; Appendix B gives the scoring rule, splits, seeds and statistical procedure, sufficient to reproduce the main claims independently of our code.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The corpus (831 adapters with mint recipes and a datasheet) and all training and evaluation code are released; an anonymized link is provided in the supplemental material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sections 2 and 3 give hyperparameters and their provenance (borrowed from the source configuration, with the regime check in Appendix A item 5); Appendix B gives splits, decoding and evaluation details; full configurations ship with the code.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Comparisons are one-sided Fisher’s exact tests against matched controls with Holm correction, plus exact McNemar tests where model and control share examples (Section 5, Appendix B); Figures 2 and 3 carry bootstrap and Wilson 95% intervals; rates are converted to counts before interpretation.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix B reports minting cost (about 550 L4 GPU-hours), training cost (about 150 H100-hours for the reported table), and that failed sweeps consumed a comparable amount.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics; it involves no human subjects, no scraped personal data, and only benign self-generated imagery.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: A broader-impact paragraph in the Limitations section discusses the dual-use asymmetry of releasing a screening tool and its corpus.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released assets are a corpus of benign visual-concept adapters and a reader for them; neither has high misuse potential requiring gated release.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All upstream assets are credited and licensed for use: FLUX.2-klein-4B, Qwen3-14B and Diffusers under Apache-2.0, ai-toolkit under MIT; the LoRAcle checkpoint declares no license and is referenced rather than redistributed (Appendix B).

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released corpus is documented in a datasheet covering composition, provenance (all images self-generated from a fixed seed), licensing (Apache-2.0) and intended use.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human-subject research was conducted, so no IRB approval was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

727 Answer: [\[Yes\]](#)
728 Justification: A language model (Qwen3-14B) is a core component of the method, described
729 in Section 2. LLM assistance was also used for coding and editing; experimental design,
730 verification and claims are the authors' own.
731 Guidelines:
732 • The answer [\[N/A\]](#) means that the core method development in this research does not
733 involve LLMs as any important, original, or non-standard components.
734 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
735 be described.